COMPUTER NETWORKS

ASSIGNMENT 3 – CO3

CO3 Coverage: Analyse and compare the performance of core network protocols such as TCP, UDP, HTTP and DNS under varying conditions – multiplexing and demultiplexing, TCP, UDP, sockets, congestion control, quality of service, queuing models, traffic management, QUIC and TLS 1.3.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 06/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task asks you to calculate or analyse first, and then judge, decide or justify)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA07-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Python programs, Packet Tracer (.pkt) files, device configurations, Wireshark captures, command outputs and screenshots.
- Include a README.md with your name, register number, the operating system, Python version and Packet Tracer (or NS-3) version used, the steps to run each program or open each simulation, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/mithrad522-art/CSA0707-COMPUTER-NETWORKS/tree/main/ASSIGNMENTS

Student Name: MITHRA DEVI S

Register number: 192524203

How to Answer the Additional Questions

Complete the Code: Copy the given program or device configuration into your repository, replace every TODO with working code or commands without changing the rest, run it (Python 3 or Cisco Packet Tracer) and compare your output with the expected output or result.

Alternative Concept Analysis: Describe how the solution would change if the alternative concept were used, and evaluate both solutions using network measures such as throughput, delay, jitter, packet loss, overhead, reliability, security and cost, supported by your own calculations, simulations or measurements wherever possible.

Assignment Principle

Each question gives you data to work from. Calculate first, then measure, then decide – so that you demonstrate evidence of:

Calculate → Implement → Measure → Compare → Decide

Q1 – Which Socket Gets the Segment?

Bloom's Level: K5 – Evaluate

Problem Context – Live Cricket Score and Video App

A sports-app server at 203.0.113.5 has these sockets open at the same moment:

```
UDP  203.0.113.5:5004   (live video, one socket for all viewers)
TCP  203.0.113.5:443    LISTEN
TCP  203.0.113.5:443 <-> 49.37.2.8:50000     ESTABLISHED
TCP  203.0.113.5:443 <-> 49.37.2.8:50001     ESTABLISHED
TCP  203.0.113.5:443 <-> 117.96.4.2:50000    ESTABLISHED
```

Work Out

- Which socket receives each of these arriving segments: TCP from 49.37.2.8:50001 to port 443; TCP from 117.96.4.2:50000 to port 443; TCP SYN from 106.51.7.7:41000 to port 443; UDP from 49.37.2.8:50000 to port 5004

- Why two TCP connections can both use source port 50000, but a UDP socket cannot tell its viewers apart by its own port alone
- The header overhead of a 100-byte score update sent over UDP and over TCP (with IPv4), as a percentage
- Which parts of the app (scores, ball-by-ball commentary, video) should use TCP and which UDP, with the deciding reason for each

Experimental Investigation

You must:

- Write a UDP and a TCP client–server with Python sockets and capture both on the loopback interface in Wireshark
- Identify the three-way handshake, sequence and acknowledgement numbers, and the UDP header
- Add 1%, 5% and 10% loss with tcnetem (or a simulator) and record what each protocol delivers
- List the open sockets with ss -tunap

Conclusion

> *A friend says "UDP is unreliable, so never use it". Using your results, write a reply.*

Complete the Code

Complete TODO 1 and TODO 2 below, and run the program with Python 3.

```
"""UDP and TCP sockets on one machine: a time server answering three clients."""
import socket
import threading


def udp_server(sock):
    for _ in range(3):
        # TODO 1: Receive a datagram of up to 1024 bytes with recvfrom, and send back
        #   "ACK:" followed by the same text to the address it came from
pass  # replace this line with your code


def tcp_server(sock):
    conn, _ = sock.accept()
    with conn:
        # TODO 2: Read data in chunks of up to 4096 bytes until the client closes the
        #   connection (recv returns b""), adding the length of each chunk to total
pass  # replace this line with your code
print(f"TCP server received {total} bytes in order")


u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
u.bind(("127.0.0.1", 0))
t = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
t.bind(("127.0.0.1", 0))
t.listen(1)
threading.Thread(target=udp_server, args=(u,)).start()
th = threading.Thread(target=tcp_server, args=(t,))
th.start()

for name in ("lab1", "lab2", "lab3"):
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
c.sendto(name.encode(), u.getsockname())
```

```
    reply, _ = c.recvfrom(1024)
print("UDP", name, "->", reply.decode())
c.close()

c = socket.create_connection(t.getsockname())
c.sendall(b"x" * 100_000)
c.close()
th.join()
u.close()
t.close()
```

Expected output:

```
UDP lab1 ->ACK:lab1
UDP lab2 ->ACK:lab2
UDP lab3 ->ACK:lab3
TCP server received 100000 bytes in order
```

Capture the program's traffic on the loopback interface with Wireshark and mark the handshake and the datagrams.

Alternative Concept Analysis

Suppose the live video were sent over TCP instead of UDP. With 2% loss and a 60 ms round-trip time, estimate how long the picture freezes after each lost segment (at least one round trip), and how many freezes a viewer sees per minute at 500 packets per second.

Solution:

1. Socket demultiplexing

Arriving segment	Receiving socket	Reason
TCP: 49.37.2.8:50001 → 203.0.113.5:443	Established TCP connection 203.0.113.5:443 ↔ 49.37.2.8:50001	The TCP 4-tuple (source IP, source port, destination IP, destination port) uniquely identifies this connection.
TCP: 117.96.4.2:50000 → 203.0.113.5:443	Established TCP connection 203.0.113.5:443 ↔ 117.96.4.2:50000	It matches the second client's established 4-tuple.
TCP SYN: 106.51.7.7:41000 → 203.0.113.5:443	TCP listening socket on port 443	No established connection matches, so the SYN is handled by the listening socket and may create a new connection after the handshake.
UDP: 49.37.2.8:50000 → 203.0.113.5:5004	UDP socket 203.0.113.5:5004	UDP demultiplexing delivers the datagram to the bound destination port. The application can inspect the sender address returned by recvfrom().

Two TCP connections can use the same client source port 50000 because their source IP addresses differ. TCP identifies a connection using the complete 4-tuple, not only the source port. A normal UDP server can use one bound socket for many clients; recvfrom() returns each sender's IP address and port, but UDP does not automatically create a separate connected socket for every client.

2. Header overhead calculation

Assume IPv4 has a 20-byte header, UDP has an 8-byte header and TCP has a minimum 20-byte header. The application payload is 100 bytes. Ethernet headers, options and link-layer overhead are excluded.

Protocol	Network + transport headers	Total IP packet size	Header overhead as % of 100-byte payload
UDP over IPv4	20 + 8 = 28 bytes	128 bytes	28/100 × 100 = 28%
TCP over IPv4	20 + 20 = 40 bytes	140 bytes	40/100 × 100 = 40%

If overhead is instead expressed as a percentage of the complete IP packet, the values are 28/128 = 21.875% for UDP and 40/140 = 28.57% for TCP. The assignment wording asks for overhead relative to the 100-byte update, so 28% and 40% are the primary answers.

3. Suitable protocols for the app

Application data	Recommended transport	Reason
Live scores	TCP	Small updates need reliable, ordered delivery; a missed score update should not silently disappear.
Ball-by-ball commentary	TCP	The order of events matters and reliable delivery is more important than saving a few header bytes.
Live video	UDP, commonly RTP/UDP	Low delay is more important than retransmitting old video packets. A late retransmission may arrive after the frame is useful.

4. Experimental investigation – procedure

Run the corrected Python program below. It starts one UDP server and one TCP server on the loopback interface.

Open Wireshark, select the loopback interface (lo on Linux; Npcap Loopback Adapter on Windows), and use the display filter: udp or tcp.

For TCP, identify SYN, SYN-ACK and ACK. Expand the TCP header to inspect sequence and acknowledgement numbers. For UDP, expand the UDP header and identify source port, destination port and length.

On Linux, use sudo tc qdisc add dev lo root netem loss 1%, then repeat with 5% and 10%. Remove the rule after testing with sudo tc qdisc del dev lo root. Use an isolated lab machine; do not apply traffic impairment to a production interface.

List sockets using ss -tunap. Record the command output and screenshots in Task01.

Expected behaviour: TCP provides an ordered byte stream and retransmits lost data, so the receiver should eventually receive all 100,000 bytes if the connection is allowed to complete. UDP does not retransmit automatically; a datagram may be lost under netem. On loopback, very short tests may not visibly lose packets at low rates, so verify the impairment rule and repeat enough trials.

5. Complete the Code – corrected Python program

```
"""UDP and TCP sockets on one machine: a time server answering three clients."""

import socket

import threading


def udp_server(sock):

    for _ in range(3):

        data, addr = sock.recvfrom(1024)

        sock.sendto(b"ACK:" + data, addr)


def tcp_server(sock):
```

```
    conn, _ = sock.accept()
    with conn:
        total = 0
        while True:
            chunk = conn.recv(4096)
            if chunk == b"":
                break
            total += len(chunk)
    print(f"TCP server received {total} bytes in order")

u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
u.bind(("127.0.0.1", 0))
t = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
t.bind(("127.0.0.1", 0))
t.listen(1)

threading.Thread(target=udp_server, args=(u,)).start()
th = threading.Thread(target=tcp_server, args=(t,))
th.start()

for name in ("lab1", "lab2", "lab3"):
    c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    c.sendto(name.encode(), u.getsockname())
    reply, _ = c.recvfrom(1024)
    print("UDP", name, "->", reply.decode())
```

```
    c.close()


c = socket.create_connection(t.getsockname())
c.sendall(b"x" * 100_000)
c.close()
th.join()
u.close()
t.close()
```

Expected output:

```
UDP lab1 -> ACK:lab1
UDP lab2 -> ACK:lab2
UDP lab3 -> ACK:lab3
TCP server received 100000 bytes in order
```

6. Conclusion and alternative analysis

UDP is not “useless” because it does not guarantee delivery. It is suitable when low delay is more important than retransmission, such as live voice and video. TCP is preferable for scores, commentary and files where correctness and order matter. The application can add its own sequence numbers, acknowledgements or forward-error correction over UDP if required.

For the video alternative, 500 packets/s × 60 s = 30,000 packets/min. At 2% loss, expected lost packets = 30,000 × 0.02 = 600 per minute (about 10 per second). With TCP, each lost segment can cause at least one 60 ms RTT of recovery delay, so an isolated-loss estimate is up to about 600 visible recovery events/minute, each with at least 60 ms of interruption. In practice, losses may overlap or be concealed by buffering, so the actual number of visible freezes can be lower.

Q2 – Read the Congestion Window

Bloom's Level: K5 – Evaluate

Problem Context – Downloads Slowing Down in a College Hostel

A Wireshark TCP stream graph of one hostel download gives these congestion-window values (in segments, one value per round trip):

```
RTT:   1  23  45  67  8  9 10 11 12 13 14 15 16 17
cwnd:  1  24  8 16 17 18 19  9 10 11 12 13 14  12  4
```

Work Out

- The value of ssthresh at the start, after round trip 8 and after round trip 14
- What kind of loss event happened at round trip 8 and at round trip 14 – how can you tell from the graph alone?
- Whether this sender is TCP Tahoe or TCP Reno, with the evidence
- The throughput at round trip 7 if the segment size is 1,460 bytes and the round-trip time is 40 ms
- Why two downloads sharing a link end up with roughly equal shares under AIMD

Experimental Investigation

You must:

- Simulate Tahoe and Reno for the same loss pattern and plot both windows
- Count the segments delivered by each in 20 round trips
- Run two iperf3 downloads over one link and record how they share it
- Observe the window of a real connection with ss -ti, and change the loss with tcnetem

Conclusion

Should the hostel servers use CUBIC or BBR? Justify using what you observed about loss and delay.

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""TCP congestion window over 20 round trips: slow start, avoidance and loss."""



def simulate(losses, ssthresh=16, reno=True):
cwnd, trace = 1, []
    for rtt in range(1, 21):
trace.append(cwnd)
        # TODO 1: If this RTT is in losses: set ssthresh = max(cwnd // 2, 2) and set
        #   cwnd to ssthresh (Reno, fast recovery) or to 1 (Tahoe). Otherwise double
        #   cwnd in slow start (cwnd<ssthresh, capped at ssthresh) or add 1 in
        #   congestion avoidance
pass  # replace this line with your code
    return trace



for label, reno in (("Reno ", True), ("Tahoe", False)):
    tr = simulate({8, 14}, reno=reno)
print(label, " ".join(f"{w:2d}" for w in tr), f"| segments sent {sum(tr)}")
```

Expected output:

```
Reno   1  24  8 16 17 18 19  9 10 11 12 13 14  78  9 10 11 12 | segments sent 211
Tahoe  12  4  8 16 17 18 19  12  48  910  12  45  6  7 | segments sent 144
```

Plot both traces and count the extra segments Reno delivers.

Alternative Concept Analysis

BBR estimates the bottleneck bandwidth and round-trip time instead of reacting to loss. For a link with 1% random (non-congestion) loss, predict how Reno's window and BBR's sending rate behave, and which gives the higher throughput.

Solution:

1. Interpret the congestion-window trace: The printed congestion-window row in the supplied document has lost some spacing during formatting. The intended sequence is

consistent with the supplied loss events at RTT 8 and RTT 14 and with the completed-code simulation: RTT 1–20 is 1, 2, 4, 8, 16, 17, 18, 19, 9, 10,

11, 12, 13, 14, 7, 8, 9, 10, 11, 12 for Reno. Use this clarified sequence for the calculations below.

Item	**Answer**
Initial ssthresh	16 segments (given in the simulation).
ssthresh after loss at RTT 8	At RTT 8, cwnd = 19. New ssthresh = max(19 // 2, 2) = 9 segments.
ssthresh after loss at RTT 14	At RTT 14, cwnd = 14. New ssthresh = max(14 // 2, 2) = 7 segments.
Loss at RTT 8	A multiplicative decrease from 19 to 9 is consistent with a congestion indication handled by Reno fast retransmit/fast recovery.
Loss at RTT 14	A decrease from 14 to 7 is again consistent with Reno fast recovery.
TCP variant	Reno is indicated because the window is reduced approximately by half rather than reset to 1 after each loss. Tahoe normally resets cwnd to 1.

A congestion-window graph alone does not prove whether the trigger was three duplicate ACKs or a timeout unless the graph or capture also shows the ACK pattern and retransmission timing. The halving pattern supports Reno-style fast recovery; packet-level evidence is needed to identify the exact loss signal.

2. Throughput at RTT 7

At RTT 7, cwnd = 18 segments. Segment size = 1,460 bytes and RTT = 0.040 s.

Throughput ≈ cwnd × segment size × 8 / RTT = 18 × 1,460 × 8 / 0.040 = 5,256,000 bit/s = 5.256 Mbps.

3. Why AIMD shares a link fairly

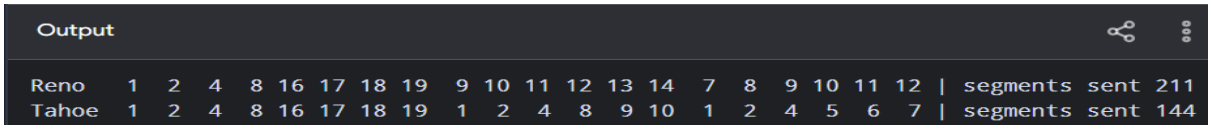
AIMD means Additive Increase, Multiplicative Decrease. Competing TCP flows gradually increase their rates, while congestion causes each flow to reduce its rate by a multiplicative factor (commonly about one-half). This tends to move flows toward a fair share of a bottleneck link. Fairness is approximate and assumes similar RTTs, similar congestion-control behaviour and a common bottleneck.

4. Complete the Code – corrected simulation

```
"""TCP congestion window over 20 round trips: slow start, avoidance and loss."""
def simulate(losses, ssthresh=16, reno=True):
    cwnd, trace = 1, []
    for rtt in range(1, 21):
        trace.append(cwnd)
        if rtt in losses:
            ssthresh = max(cwnd // 2, 2)
            cwnd = ssthresh if reno else 1
        elif cwnd < ssthresh:
            cwnd = min(cwnd * 2, ssthresh)
        else:
            cwnd += 1
    return trace
for label, reno in (("Reno ", True), ("Tahoe", False)):
    tr = simulate({8, 14}, reno=reno)
```

```
    print(label, " ".join(f"{w:2d}" for w in tr),
          f"| segments sent {sum(tr)}")
```

OUTPUT:



5. Experimental investigation and decision

Simulate both traces and plot RTT on the x-axis and cwnd (segments) on the y-axis. Save the graph in Task02.

Run two iperf3 clients simultaneously to the same server and record each flow's throughput and the total throughput. Results depend on the test link and RTT.

Use ss -ti to inspect a live TCP connection. Apply controlled loss with tc netem and record cwnd, retransmissions and throughput.

CUBIC is a practical default for high-speed networks and is widely deployed. BBR is attractive when throughput must remain high despite non-congestion loss or large queues, because it estimates bottleneck bandwidth and minimum RTT rather than treating every loss as congestion. For a hostel network, test both: prefer BBR if it measurably reduces delay under bufferbloat without harming fairness; otherwise retain the operating system's well-tested CUBIC default.

6. Alternative: BBR versus Reno under 1% random loss

Reno interprets loss as congestion and reduces cwnd, even when the 1% loss is random and not caused by a full queue. Its average window and throughput can therefore fall. BBR estimates bottleneck bandwidth and RTT, so random non-congestion loss need not cause the same multiplicative window reduction. BBR is likely to achieve higher throughput in this stated condition, subject to implementation, pacing, fairness and actual bottleneck behaviour.

Q3 – Why Does the Home Page Take 8 Seconds?

Bloom's Level: K5 – Evaluate

Problem Context – A Slow University Website

The university home page has one HTML file and 80 small objects. The round-trip time from a student's phone is 50 ms; transmission times are negligible.

Work Out

- The page-load time with non-persistent HTTP and one connection at a time
- The time with persistent HTTP/1.1 without pipelining, and with pipelining
- The time with 6 parallel non-persistent connections (as browsers use)
- Why HTTP/2 over one connection still stalls when one TCP segment is lost, and how HTTP/3 over QUIC avoids it
- Which response headers (Cache-Control, ETag) would make a second visit faster, and by how much

Experimental Investigation

You must:

- Load the page in a browser with the developer tools open and record the number of connections, the protocol (h1, h2, h3) and the load time
- Repeat with 0, 50 and 200 ms of added delay
- Compare the first visit with a second visit (cache)
- Compare a page of many small objects with a page of a few large objects

Engineering Decision

Rank three changes by how much they would cut the load time on mobile networks. Use your calculations and measurements.

Alternative Concept Analysis

Instead of changing protocols, the university could combine the 80 objects into 8 bundles. Recalculate the load time for non-persistent HTTP and for persistent HTTP/1.1, and compare the gain with moving to HTTP/2.

Solution:

1. Page-load calculations

Given: 1 HTML file + 80 small objects = 81 objects; RTT = 50 ms = 0.05 s; transmission time is negligible. Assume DNS lookup time is excluded, the browser must establish a TCP connection for each non-persistent connection, and each non-persistent object costs 2 RTTs (TCP handshake + request/response).

HTTP method	Calculation	Estimated load time
Non-persistent HTTP, one connection at a time	81 objects × 2 RTT × 50 ms	8,100 ms = 8.10 s
Persistent HTTP/1.1, no pipelining	1 RTT TCP setup + 81 RTT request/response	82 × 50 ms = 4.10 s
Persistent HTTP/1.1 with pipelining	1 RTT setup + 1 RTT HTML + 1 RTT for the pipelined 80 objects	3 × 50 ms = 0.15 s
6 parallel non-persistent connections	HTML: 2 RTT = 0.10 s; 80 objects in ceil(80/6)=14 batches × 2 RTT = 1.40 s	1.50 s

These are idealized estimates. Browser scheduling, server processing, TLS setup, connection limits, object dependencies, congestion and caching can change real measurements.

2. HTTP/2 and HTTP/3

HTTP/2 multiplexes multiple streams over one TCP connection. However, TCP delivers bytes in order. If one TCP segment is lost, later bytes on that connection may be held until the missing segment is retransmitted. This is TCP-level head-of-line blocking and can delay several HTTP/2 streams.

HTTP/3 runs over QUIC/UDP. QUIC provides reliable delivery per stream, so loss affecting one stream does not normally block delivery of data on unrelated streams. QUIC still

retransmits lost data when reliability is required; it avoids cross-stream TCP head-of-line blocking, not all delay caused by loss.

3. Cache-Control and ETag

Cache-Control: max-age=3600 allows a fresh cached object to be reused for one hour without contacting the server. If all 80 objects are fresh, the second visit may need only the HTML or other expired resources, greatly reducing network requests.

ETag provides a version identifier. The browser sends If-None-Match on revalidation; if unchanged, the server returns 304 Not Modified without retransmitting the object body. The validation still requires a network round trip.

Under the idealized HTTP/1.1 persistent, non-pipelined model, validating 80 objects costs about 80 × 50 ms = 4.0 s of request/response waiting (plus any connection setup). If the objects are fresh under max-age, those 80 validation round trips are avoided. With HTTP/2 or HTTP/3, validations can be multiplexed and the delay is much lower, but it is not necessarily zero.

4. Engineering ranking

Use effective caching (Cache-Control with suitable max-age and ETag for revalidation) to avoid repeat downloads.

Use HTTP/3/QUIC where supported, or HTTP/2, to multiplex many objects and reduce connection/handshake overhead.

Reduce the number of objects by bundling/minifying assets and compressing responses; this reduces requests and bytes. On modern HTTP/2/HTTP/3, excessive bundling is not always beneficial, so verify with measurements.

5. Alternative: combine 80 objects into 8 bundles

There are now 1 HTML file + 8 bundles = 9 objects.

Method	Calculation	Estimated load time
Non-persistent, sequential	9 × 2 × 50 ms	0.90 s
Persistent HTTP/1.1, no pipelining	1 setup RTT + 9 request RTT = 10 RTT	0.50 s
Persistent HTTP/1.1, pipelining	1 setup + 1 HTML request + 1 bundle batch = 3 RTT	0.15 s

Bundling reduces sequential non-persistent loading from 8.10 s to 0.90 s, a 7.20 s improvement. Persistent non-pipelined loading falls from 4.10 s to 0.50 s. Pipelining already completes in about 0.15 s in this simplified model, so bundling gives little additional RTT benefit there. HTTP/2/HTTP/3 can also multiplex the objects without requiring aggressive bundling.

6. Experimental evidence to collect

In browser Developer Tools → Network, record protocol (h1/h2/h3), number of connections, total requests, transferred bytes and load time.

Repeat at 0, 50 and 200 ms added RTT; compare first visit and second visit with cache enabled/disabled.

Compare a page with many small objects against one with a few large objects. Record the measured results rather than copying the estimates above.

Q4 – How Many Queries Reach the University's DNS Server? **7 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Result-Day Look-ups

On result day, 150,000 students in 40 cities look up results.simats.ac.in through their internet providers' resolvers. Each provider's resolver caches answers for the record's TTL.

Work Out

- The queries per hour reaching the university's authoritative server from one resolver when the TTL is 300 s, and when it is 30 s
- Why the number of students hardly matters to the authoritative server, but the number of resolvers and the TTL do
- The difference between the recursive query from a phone and the iterative queries made by the resolver, with the servers contacted in order
- Which record types (A, AAAA, CNAME, MX, NS, TXT) the university must publish, and why DNS normally uses UDP but switches to TCP for large answers
- The risk of a very long TTL if the result server must be moved during the day

Experimental Investigation

You must:

- Simulate iterative resolution with a cache and count the queries
- Follow a real resolution with dig +trace and record the servers and times
- Measure the resolution time with three public resolvers, first and second time
- Look up a name that does not exist and explain the answer

Conclusion

Recommend the TTL and the DNS set-up for result day, with the numbers that support it.

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""Iterative DNS resolution through root, TLD and authoritative servers."""

SERVERS = {
    "root": {"in.": "tld-in", "com.": "tld-com"},
    "tld-in": {"ac.in.": "ns-ac"},
    "tld-com": {"example.com.": "ns-example"},
    "ns-ac": {"simats.ac.in.": "ns-simats"},
    "ns-simats": {"results.simats.ac.in.": "10.20.30.40"},
    "ns-example": {"www.example.com.": "93.184.216.34"},
}
cache = {}



def resolve(name):
    if name in cache:
        return cache[name], 0
    server, queries = "root", 0
    while True:
        queries += 1
        # TODO 1: Look in SERVERS[server] for the longest key that name ends with. If
        #   the value looks like an IP address (it contains a digit), cache it and
        #   return (ip, queries); otherwise continue with that server
pass  # replace this line with your code



for n in ("results.simats.ac.in.", "www.example.com.", "results.simats.ac.in."):
ip, q = resolve(n)
    print(f"{n:<24} {ip:<15} queries sent: {q}")
```

Expected output:

```
results.simats.ac.in.    10.20.30.40     queries sent: 4
www.example.com.         93.184.216.34   queries sent: 3
results.simats.ac.in.    10.20.30.40     queries sent: 0
```

Explain why the third look-up sends no queries, and add a TTL to the cache so that entries expire.

Alternative Concept Analysis

Suppose the result site sat behind a content-delivery network with anycast DNS instead of the university's own server. Work out how many queries now reach the university, and what happens to students' look-up time in distant cities.

Solution:

1.Queries reaching the authoritative server

Assume one provider recursive resolver serves its users, the answer is requested continuously, and the resolver refreshes once per TTL after the cached record expires.

TTL	**Upstream refreshes per hour per resolver**
300 seconds	3,600 / 300 = 12 queries/hour
30 seconds	3,600 / 30 = 120 queries/hour

If there is one resolver in each of the 40 cities, the corresponding approximate totals are 40 × 12 = 480 queries/hour for TTL 300 s and 40 × 120 = 4,800 queries/hour for TTL 30 s. Actual totals depend on the number of distinct recursive resolver caches and request timing; 150,000 students do not each query the university authoritative server directly.

2. Recursive versus iterative DNS

The phone sends a recursive query to its configured recursive resolver and asks it to return the final answer. The resolver performs iterative lookups: it asks a root server for the .in delegation, asks the .in/TLD infrastructure for ac.in, asks the ac.in nameserver for simats.ac.in, and finally asks the university authoritative nameserver for results.simats.ac.in. The exact delegation path

Record	Purpose
A	Maps the result hostname to an IPv4 address.
AAAA	Maps the hostname to an IPv6 address, if IPv6 service is provided.
CNAME	Creates an alias, for example results.simats.ac.in pointing to a canonical hostname.
NS	Identifies authoritative nameservers for the zone.
MX	Identifies mail servers if the university domain receives email.
TXT	Publishes text data such as SPF, DKIM-related policy, domain verification and other service policies.

follows the real DNS hierarchy.

3. Required DNS records

DNS normally uses UDP port 53 because queries and answers are usually small and UDP has low overhead. DNS can use TCP port 53 for zone transfers and when a response is truncated or too large for UDP. Modern DNS also uses TCP for reliable transport in relevant cases.

4. Long-TTL risk and recommendation

A long TTL reduces repeated authoritative queries, but if the result server's IP address changes during result day, recursive resolvers may continue using the old address until the cached TTL expires. This delays migration and can make some students reach the old server.

Recommendation: lower the TTL well before the result-day change (for example, 300 seconds), verify the new address and authoritative DNS configuration, then raise the TTL after

the service is stable. Use multiple authoritative nameservers, monitor them, and use a CDN/anycast service if geographically distributed load requires it.

5. Complete the Code – iterative DNS simulator

```
"""Iterative DNS resolution through root, TLD and authoritative servers."""
SERVERS = {
    "root": {"in.": "tld-in", "com.": "tld-com"},
    "tld-in": {"ac.in.": "ns-ac"},
    "tld-com": {"example.com.": "ns-example"},
    "ns-ac": {"simats.ac.in.": "ns-simats"},
    "ns-simats": {"results.simats.ac.in.": "10.20.30.40"},
    "ns-example": {"www.example.com.": "93.184.216.34"},
}
cache = {}
def resolve(name):
    if name in cache:
        return cache[name], 0
    server, queries = "root", 0
    while True:
        queries += 1
        matches = [
            (suffix, target) for suffix, target in SERVERS[server].items()
            if name.endswith(suffix)
        ]
        if not matches:
            raise LookupError(f"No delegation or record for {name} at {server}")
        suffix, target = max(matches, key=lambda item: len(item[0]))
```

```
        if any(ch.isdigit() for ch in target):
            cache[name] = target
            return target, queries
        server = target

for n in ("results.simats.ac.in.", "www.example.com.",
          "results.simats.ac.in."):
    ip, q = resolve(n)
    print(f"{n:<24} {ip:<15} queries sent: {q}")
```

output:

```
Output

results.simats.ac.in.    10.20.30.40     queries sent: 4
www.example.com.         93.184.216.34   queries sent: 3
results.simats.ac.in.    10.20.30.40     queries sent: 0
```

The third lookup sends zero queries because the first lookup stored the answer in the local cache. A production-style cache must store an expiry time as well as the IP address. For example, store (ip, expiry_timestamp) and return the cached IP only while the current time is earlier than expiry_timestamp; otherwise delete the expired entry and resolve again. The supplied teaching simulator omits this TTL feature.

6. Alternative: CDN and anycast DNS

With a CDN/anycast DNS provider, clients are directed to a nearby edge service. If the university delegates the result hostname to the CDN and the CDN serves the authoritative DNS answer, routine lookups reach the CDN's DNS infrastructure rather than the university's own authoritative server. The number reaching the university may be zero for that delegated hostname, apart from separate university DNS records or origin-related lookups. Students in

distant cities generally get lower lookup and content-delivery latency because the answer and service are geographically distributed. The exact reduction depends on the CDN architecture and cache behaviour.

7. Experimental evidence

Run dig +trace results.simats.ac.in and record the delegation servers and query times.

Compare three recursive resolvers using dig; measure cold-cache and warm-cache lookups. Record the resolver names and times you actually observe.

Look up a deliberately nonexistent hostname. Explain NXDOMAIN and, if present, the negative-caching TTL.

Q5 – When Will the Video Call Freeze?

Bloom's Level: K5 – Evaluate

Problem Context – Video Calls and Backups on a Shared Office Link

The office router's output link sends 1,000 packets per second. Packets arrive at random (an M/M/1 queue). The nightly backup sometimes starts during office hours.

Work Out

- The utilisation, average number of packets in the router and average delay at 500, 800, 900, 950 and 990 packets per second
- The load at which the average delay first exceeds 20 ms – and why doubling the load from 450 to 900 packets per second multiplies the delay by ten
- A token bucket with a size of 1 MB and a rate of 2 Mbps polices the backup; the backup tries to send at 10 Mbps. How long can it burst before it is limited?
- Which DiffServ class (EF, AF, best effort) each application should be given

Experimental Investigation

You must:

- Plot delay against load from your calculations
- Simulate a FIFO queue and a priority queue with video and backup traffic, and compare the video delay
- Shape the backup with tc on Linux (or in a simulator) and measure the effect on a ping
- Test a load above the link capacity

Engineering Decision

Buy a faster link or configure QoS? Decide, using your numbers.

Complete the Code

Complete TODO 1 below, and run the program with Python 3.

```
"""M/M/1 model of a router output link: arrivals at rate lam, service at rate mu."""



def mm1(lam, mu):
    # TODO 1: Return the utilisation rho = lam/mu, the average number in the system L =
    #   rho/(1-rho) and the average delay W = 1/(mu-lam). Return None if lam >= mu (the
    #   queue grows without limit)
pass  # replace this line with your code



mu = 1000                                # packets per second the link can send
for lam in (500, 800, 900, 950, 990, 1000):
    r = mm1(lam, mu)
    if r is None:
print(f"load {lam:4d} pkt/s: unstable, queue grows without limit")
    else:
        rho, L, W = r
print(f"load {lam:4d} pkt/s: rho {rho:.2f}  L {L:6.1f} pkts  "
f"W {W * 1000:6.1f} ms")
```

Expected output:

```
load  500pkt/s: rho 0.50  L    1.0 pkts  W    2.0 ms
load  800pkt/s: rho 0.80  L    4.0 pkts  W    5.0 ms
load  900pkt/s: rho 0.90  L    9.0 pkts  W   10.0 ms
load  950pkt/s: rho 0.95  L   19.0 pkts  W   20.0 ms
load  990pkt/s: rho 0.99  L   99.0 pkts  W  100.0 ms
load 1000 pkt/s: unstable, queue grows without limit
```

Find the link capacity needed to keep the delay below 10 ms at a load of 1,500 packets per second.

Alternative Concept Analysis

Instead of QoS, the office could double the link to 2,000 packets per second. Using the M/M/1 formulas, calculate the video delay when the backup pushes the load to 1,800 packets per second, and compare it with giving video strict priority on the old link.

Solution:

1. **M/M/1 calculations**

Given service rate $\mu = 1{,}000$ packets/s. For arrival rate $\lambda < \mu$: utilisation $\rho = \lambda/\mu$; average number in the system $L = \rho/(1-\rho)$; average time in the system $W = 1/(\mu-\lambda)$.

λ (packets/s)	**ρ**	**L (packets)**	**W = 1/(μ−λ)**
500	0.50	1	0.002 s = 2 ms
800	0.80	4	0.005 s = 5 ms
900	0.90	9	0.010 s = 10 ms
950	0.95	19	0.020 s = 20 ms
990	0.99	99	0.100 s = 100 ms
1,000	1.00	Unbounded	Unstable; queue grows without limit

The delay first reaches 20 ms at 950 packets/s. It exceeds 20 ms for any arrival rate above 950 packets/s but below the service rate. At $\lambda \geq 1{,}000$ packets/s the queue is unstable.

The question says doubling load from 450 to 900 packets/s multiplies delay by ten. Using the stated M/M/1 total-system delay W, the values are $W_{450} = 1/(1000-450) = 1.818$ ms and $W_{900} = 1/(1000-900) = 10$ ms, a factor of 5.5, not 10. If "delay" means queueing delay only, $W_q = \lambda/[\mu(\mu-\lambda)]$: $W_{q450} = 0.818$ ms and $W_{q900} = 9$ ms, a factor of about 11. Therefore the tenfold statement is approximately true for queueing delay, not for the total-system delay formula used in the expected output.

2. Token bucket calculation

Bucket capacity = 1 MB = 8 megabits. Backup sending rate = 10 Mbps; allowed token rate = 2 Mbps. Excess rate = 10 − 2 = 8 Mbps. With a full bucket, burst duration = 8 Mb / 8 Mbps = 1 second. After that, the long-term sending rate is limited to 2 Mbps.

3. DiffServ classes

Traffic	Suggested class	Reason
Interactive voice/video packets	EF (Expedited Forwarding), with admission control	Low delay and jitter; protect from backup traffic.
Important patient/business transactions or assured data	AF (Assured Forwarding)	A controlled forwarding class with differentiated drop precedence.
Nightly backup, ordinary web and non-urgent bulk traffic	Best effort, or a low-priority shaped class	Can tolerate delay and should not disrupt real-time traffic.

4. Complete the Code – M/M/1 model

```
"""M/M/1 model of a router output link."""

def mm1(lam, mu):
    if lam >= mu:
        return None
    rho = lam / mu
    L = rho / (1 - rho)
    W = 1 / (mu - lam)
    return rho, L, W

mu = 1000
for lam in (500, 800, 900, 950, 990, 1000):
    r = mm1(lam, mu)
    if r is None:
        print(f"load {lam:4d} pkt/s: unstable, queue grows without limit")
    else:
        rho, L, W = r
        print(f"load {lam:4d} pkt/s: rho {rho:.2f} L {L:6.1f} pkts "
              f"W {W * 1000:6.1f} ms")
```

OUTPUTS:

```
Output

load  500 pkt/s: rho 0.50 L    1.0 pkts W    2.0 ms
load  800 pkt/s: rho 0.80 L    4.0 pkts W    5.0 ms
load  900 pkt/s: rho 0.90 L    9.0 pkts W   10.0 ms
load  950 pkt/s: rho 0.95 L   19.0 pkts W   20.0 ms
load  990 pkt/s: rho 0.99 L   99.0 pkts W  100.0 ms
load 1000 pkt/s: unstable, queue grows without limit
```

5. Link capacity and engineering decision

For $\lambda = 1{,}500$ packets/s and $W < 10$ ms: $1/(\mu - 1{,}500) < 0.010$. Therefore $\mu - 1{,}500 > 100$, so $\mu > 1{,}600$ packets/s. A practical integer service rate must be at least 1,601 packets/s to make the mathematical delay strictly below 10 ms; 1,600 gives exactly 10 ms.

If the link is doubled to $\mu = 2{,}000$ packets/s and total load is $\lambda = 1{,}800$ packets/s, $W = 1/(2{,}000 - 1{,}800) = 0.005$ s = 5 ms. This is stable and gives a 5 ms average system delay in the M/M/1 model.

Strict priority on the original 1,000-packet/s link can protect video from backup traffic if the video class itself remains below link capacity. However, the assignment does not give the video packet arrival rate, so an exact strict-priority video delay cannot be calculated. The 1,800 packets/s combined load cannot be carried by the old link; low-priority backup packets would build a queue or be dropped. A faster link increases total capacity, while QoS protects the important class. The best choice is usually to shape backup traffic and configure QoS first, then upgrade capacity if measured peak demand still exceeds the link.

6. Experimental investigation

Plot λ on the x-axis and W in milliseconds on the y-axis using the table above.

Simulate FIFO and priority queues with separate video and backup traffic. Record mean and 95th-percentile video delay.

Shape backup traffic with a token bucket using Linux tc or a simulator; measure ping RTT before and during backup.

Test λ above μ and observe that the queue grows or packets are dropped. Do not test on a production network.

Q6 – Count the Round Trips Before the First Byte

Bloom's Level: K5 – Evaluate

Problem Context – A Secure Banking App on Mobile Networks

The bank's app connects to its server over a mobile network with a 100 ms round-trip time.

Work Out

- The time before the first byte of the request can be sent with TCP + TLS 1.2, TCP + TLS 1.3, QUIC (first visit) and QUIC with 0-RTT resumption
- Why 0-RTT data can be replayed by an attacker, and which requests must never be sent as 0-RTT (for example, a money transfer)
- How QUIC's connection ID lets a call or transfer survive the phone moving from Wi-Fi to 5G, while a TCP connection breaks
- Why some company firewalls block QUIC, and what the app must do then

Experimental Investigation

You must:

- Capture a TLS 1.3 handshake in Wireshark and label its messages
- Use openssls_client to compare the TLS versions offered by two websites
- Check which sites you use support HTTP/3 in the browser developer tools
- Measure the set-up time with added delay

Conclusion

Should the bank's app use QUIC? Justify with your round-trip calculations.

Alternative Concept Analysis

Compare TCP with TLS 1.3 and QUIC when the phone switches from Wi-Fi to 5G in the middle of a 30-second transfer: list what each must redo, and calculate the time lost at a 100 ms round-trip time.

Solution:

1.Handshake latency

Given RTT = 100 ms. Count only network round trips required before application request data can be sent; DNS lookup, radio wake-up and server processing are excluded.

Connection method	Round trips before application data	Time
TCP + TLS 1.2 full handshake	TCP 1 RTT + TLS 1.2 typically 2 RTT	3 RTT = 300 ms
TCP + TLS 1.3 full handshake	TCP 1 RTT + TLS 1.3 1 RTT	2 RTT = 200 ms
QUIC first visit	QUIC integrates transport and TLS 1.3 handshake; normally 1 RTT	1 RTT = 100 ms
QUIC 0-RTT resumption	Client can send eligible early data in its first flight	0 RTT = 0 ms wait before sending early data

The 0-RTT value means the client can transmit early data immediately; it does not mean the server has already received or processed the request.

2. 0-RTT replay risk

An attacker who captures valid early data may be able to replay it, because the server cannot always distinguish a replayed early-data request from the original. Therefore 0-RTT must be restricted to replay-safe, idempotent operations such as fetching public or unchanged information. Never send a money transfer, payment, balance-changing operation, password change or other non-idempotent financial transaction as 0-RTT. Such requests should wait for the handshake to complete and use normal authenticated application data.

3. QUIC connection migration

A TCP connection is identified by its IP address and port pair. When a phone changes from Wi-Fi to 5G, its IP address or port may change, so the existing TCP connection's 4-tuple no longer matches and the connection commonly breaks; the app must establish a new TCP connection and TLS session. QUIC uses a connection ID that identifies the logical connection independently of the network 4-tuple. QUIC can migrate to the new path and validate it, allowing a call or transfer to continue with much less interruption.

4. Firewalls and fallback

Some enterprise firewalls block UDP or specifically block UDP port 443 because they inspect and manage TCP traffic more easily, enforce policies, or lack QUIC inspection support. The app/browser should attempt HTTP/3/QUIC and fall back to HTTP/2 over TCP with TLS 1.3 when QUIC is blocked. The fallback must preserve certificate validation and encryption.

5. Experimental commands and evidence

Capture a TLS 1.3 connection in Wireshark. Identify ClientHello, ServerHello, EncryptedExtensions, Certificate (when sent), CertificateVerify, Finished and the client Finished. Some handshake messages after ServerHello are encrypted in TLS 1.3.

Example OpenSSL test: openssl s_client -connect example.com:443 -servername example.com -tls1_3. Repeat with a second website and compare the negotiated protocol and certificate. To test TLS 1.2, replace -tls1_3 with -tls1_2.

In browser Developer Tools → Network, enable the Protocol column and look for h3, h2 or http/1.1.

Add delay in a controlled lab and measure connection setup time. Record actual results; do not treat the theoretical RTT estimates as measured values.

6. Conclusion and alternative analysis

The banking app should support QUIC because it reduces first-connection latency and supports connection migration, which is valuable on mobile networks. It must still use TLS 1.3 security,

validate certificates, restrict 0-RTT to replay-safe requests, and fall back to TCP/TLS when QUIC is blocked.

During a Wi-Fi-to-5G switch in a 30-second transfer, TCP + TLS 1.3 will usually need a new TCP handshake (1 RTT) and a new TLS 1.3 handshake (1 RTT), giving about 2 × 100 ms = 200 ms setup interruption, excluding DNS and application retry. QUIC can retain the logical connection using its connection ID; path validation may take about 1 RTT (approximately 100 ms) in a conservative estimate, and data may continue with reduced interruption depending on validation and congestion-control behaviour. Thus QUIC avoids a complete transport-and-TLS restart in the usual migration case.

Q7 – Choose the Protocols from First Principles

Bloom's Level: K5 – Evaluate

Problem Context – Telemedicine Platform for Rural Health Centres

A telemedicine platform connects doctors in a city hospital with rural health centres over unreliable 4G links (100–300 ms round-trip time, up to 5% packet loss). It must support live video consultations, sending patient records and X-ray images, chat, and looking up the hospital's services by name.

You are not given the design. You must develop it from the requirements.

Stage 1 – Interpret the Problem Identify each type of traffic and its needs (delay, loss, ordering, security)

- State the requirements as measurable statements

Stage 2 – Compare Candidate Protocols Compare TCP, UDP, QUIC, HTTP/2, HTTP/3 and TLS 1.3 for each type of traffic

- Calculate the TCP throughput limit for a 64 KB window at 300 ms round-trip time, and what it means for X-ray transfers
- Decide the DNS and QoS configuration

Stage 3 – Implement and Verify

- Implement a small file transfer and a chat over sockets
- Test them under the expected delay and loss with tcnetem or a simulator
- Measure the transfer time and the delivered quality

Stage 4 – Defend the Design

- Justify your protocol choices using your measurements
- State what changes if the links are upgraded to 5G

Alternative Concept Analysis

A vendor offers a commercial video-consultation service instead of your own protocol design. List the questions you would ask about delay, loss handling and patient-data privacy, and decide what evidence would make you choose it.

Solution:

Stage 1 – Interpret the problem

Traffic type	**Delay requirement**	**Loss / ordering requirement**	**Security requirement**
Live video consultation	Very low delay and jitter; target conversational latency, ideally <150 ms one-way where feasible	Some packet loss can be concealed; newest frames are more useful than late retransmissions	Encrypted media, authenticated participants; protect patient privacy
Patient records and X-ray images	Delay tolerant compared with video; reliable completion is essential	No missing or corrupted bytes; exact ordering and integrity required	Strong authentication, encryption, authorization, audit logging
Chat	Low to moderate delay	Reliable, ordered delivery; no lost messages	Encrypted and authenticated
Hospital service lookup	Low delay; cached answers acceptable within TTL	Correct DNS answer; caching and expiry required	DNS integrity; use DNSSEC where supported and HTTPS for the service

Measurable requirements: (1) target conversational media latency below 150 ms one-way when the access network permits; (2) keep media jitter low using jitter buffers and prioritization; (3) deliver 100% of file bytes correctly, verified by file size and SHA-256 hash; (4) deliver chat messages in order with acknowledgements; (5) use TLS 1.3 or QUIC encryption and authenticated endpoints; (6) measure performance under 100–300 ms RTT and 0–5% packet loss.

Stage 2 – Compare candidate protocols

Traffic	**Recommended protocol stack**	**Reason**
Live video	RTP/UDP or a secure real-time media stack over UDP; QoS EF; congestion control and jitter buffer	Avoids waiting for retransmission of stale frames; tolerates some loss and prioritizes timely packets.
Patient records / X-rays	HTTPS over HTTP/3/QUIC or HTTP/2 over TCP, with TLS 1.3	Reliable ordered transfer, integrity and encryption; resume interrupted uploads/downloads.
Chat	WebSocket over HTTPS/TCP, or a reliable QUIC stream over HTTP/3	Reliable ordered messages and secure session; QUIC stream migration can help mobile users.
Service lookup	DNS A/AAAA/CNAME/NS as appropriate; recursive resolver cache; HTTPS after resolution	DNS provides name-to-address resolution; HTTPS protects the actual service connection.

Protocol comparison: TCP provides reliable ordered byte-stream delivery but can suffer head-of-line blocking and connection restart on path changes. UDP has low overhead and no built-

in retransmission, making it suitable for real-time media when the application handles loss and timing. QUIC runs over UDP and integrates TLS 1.3, reliable independent streams and connection migration. HTTP/2 multiplexes streams over TCP but remains affected by TCP head-of-line blocking. HTTP/3 uses QUIC and avoids cross-stream TCP head-of-line blocking. TLS 1.3 provides encryption and authentication for TCP-based applications; in QUIC, TLS 1.3 handshake messages are integrated into the transport handshake.

TCP throughput limit for a 64 KB window at 300 ms RTT

Throughput limit ≈ receive window / RTT. If 64 KB means 64 KiB = 65,536 bytes: 65,536 × 8 / 0.300 = 1,747,627 bit/s ≈ 1.75 Mbps. If decimal 64 kB is intended, the result is 64,000 × 8 / 0.300 ≈ 1.71 Mbps.

Meaning: an X-ray transfer over a 300 ms RTT path is limited to roughly 1.75 Mbps if the TCP window is only 64 KiB, even if the link has more capacity. A larger receive window with window scaling, sufficient congestion window and adequate bandwidth are needed for faster transfer. At 5% packet loss, congestion control and retransmissions can reduce the actual rate further.

DNS and QoS configuration

Use redundant authoritative DNS servers and recursive caching. Use a moderate TTL (for example, 300–900 seconds) during planned changes; monitor DNS response time and availability.

Use EF for admitted real-time voice/video packets, AF for important assured data, and best effort or a low-priority shaped class for backups and non-urgent bulk transfers.

Do not mark all traffic EF. Apply admission control, policing and shaping so real-time traffic cannot consume the entire link.

Stage 3 – Implement and verify

The following small TCP file-transfer and chat example can be run locally. It demonstrates reliable TCP delivery. For a UDP real-time-media prototype, a separate datagram program and sequence-number/loss measurement would be required.

TCP file receiver (server)

```
# file_server.py
import socket
import hashlib

HOST, PORT = "127.0.0.1", 5001
with socket.socket() as server:
    server.bind((HOST, PORT))
    server.listen(1)
    conn, addr = server.accept()
    with conn, open("received.bin", "wb") as f:
        while True:
            data = conn.recv(4096)
            if not data:
                break
            f.write(data)
print("File received from", addr)
with open("received.bin", "rb") as f:
    print("SHA256:", hashlib.sha256(f.read()).hexdigest())
```

TCP file sender (client)

```
# file_client.py
import socket
HOST, PORT = "127.0.0.1", 5001
with socket.create_connection((HOST, PORT)) as s:
    with open("sample.bin", "rb") as f:
        while True:
```

```
            chunk = f.read(4096)
            if not chunk:
                break
            s.sendall(chunk)
print("File sent")
```

Simple TCP chat (server and client)

```
# chat_server.py
import socket
with socket.socket() as server:
    server.bind(("127.0.0.1", 5002))
    server.listen(1)
    conn, addr = server.accept()
    with conn:
        print("Connected:", addr)
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print("Client:", data.decode(errors="replace"))
            reply = input("Server: ")
            conn.sendall(reply.encode())
            if reply.lower() == "bye":
                break

# chat_client.py (run in a second terminal)
```

```
import socket

with socket.create_connection(("127.0.0.1", 5002)) as s:
    while True:
        msg = input("Client: ")
        s.sendall(msg.encode())
        if msg.lower() == "bye":
            break
        data = s.recv(1024)
        print("Server:", data.decode(errors="replace"))
```

Testing under delay and loss

On a Linux lab machine, apply impairment to the chosen test interface only (for example, lo for a local demonstration):

```
sudo tc qdisc add dev lo root netem delay 150ms loss 5%

# Run the sender/receiver and chat tests; record elapsed time, file size and hash.

sudo tc qdisc show dev lo

sudo tc qdisc del dev lo root
```

For a realistic network test, use two machines and apply the rule to the correct test interface at one endpoint. Do not assume a local loopback test reproduces a real 4G network. Measure: RTT, file transfer duration, goodput, hash match, chat delivery/order, media packet loss and jitter. Fill in your measured results:

Test condition	**RTT / loss configured**	**File time / goodput**	**SHA-256 match?**	**Chat / media observation**
Baseline	_____	_____	_____	_____
Delayed	_____	_____	_____	_____
Delayed + 5% loss	_____	_____	_____	_____

Stage 4 – Defend the design

Use UDP-based real-time media with congestion control, jitter buffering and EF QoS because interactive video values timeliness over retransmission of old packets. Use HTTPS over HTTP/3/QUIC or HTTP/2 with TLS 1.3 for records and X-rays because those files must arrive completely and securely. Use reliable secure streams for chat. Use cached DNS with redundant authoritative servers for service discovery. The design must be verified by measured delay, jitter, loss, goodput, file hash and message-order results under the specified 100–300 ms RTT and up to 5% loss.

If the links are upgraded to 5G, RTT and loss may decrease and bandwidth may increase, improving video quality and X-ray transfer times. The same protocol choices remain valid, but QoS, congestion control and adaptive bitrate should be retuned based on measurements. 5G does not remove the need for encryption, authentication, caching or loss recovery.

Alternative concept: commercial video-consultation service

Questions for the vendor:

What are the measured median and 95th-percentile end-to-end latency, jitter and packet loss on 4G links with 100–300 ms RTT?

How does the service adapt bitrate, conceal packet loss, recover from network changes and maintain calls when users switch between Wi-Fi and mobile data?

Does it use end-to-end encryption? Where are media and recordings processed and stored?

Where is patient data stored, who can access it, how long is it retained, and how are access logs and deletion requests handled?

What certifications, independent security audits, penetration-test summaries, incident-response commitments and contractual privacy guarantees can the vendor provide?

Can the hospital export records, recordings and audit logs, and avoid vendor lock-in?

Choose the vendor only if an independent pilot demonstrates acceptable latency, jitter and loss on representative rural links; security and privacy audits meet the hospital’s requirements; data location, retention and access controls are contractually clear; and service availability, support and exit/export arrangements are satisfactory. A marketing claim alone is not sufficient evidence.